

Guía de defensa — Examen Final MCC225

Proyecto 5: Evaluación del razonamiento visio-lingüístico composicional en CLIP mediante Winoground

Niels Victor Pacheco Barrios

Período 2026-1

Índice

Cómo usar esta guía	2
Los números que hay que saber de memoria	2
Tabla principal — CLIP ViT-B/32 (laion2b), 400 pares oficiales	2
Los otros dos checkpoints	2
Desglose por tag (ViT-B/32)	3
Prueba de ceguera	3
Retrieval vs composición (galería completa de 800)	3
Potencia estadística	3
Empates y márgenes (ViT-B/32)	3
Síntesis inicial — 2 minutos (Sec. 2.1, vale 1 punto)	3
Sec. 11.4 — Preguntas conceptuales, metodológicas y críticas	4
1. Defina text score, image score y group score sin leer el código	4
2. ¿Por qué el azar del group score es 1/6?	4
3. ¿Cómo puede coexistir Recall@5 alto con group score bajo?	5
4. ¿Qué mide la prueba de ceguera y qué no mide?	5
5. ¿Por qué la permutación de imágenes no demuestra por sí sola razonamiento composicional?	6
6. ¿Qué tipos de errores dominan y por qué son difíciles para un dual encoder?	6
7. ¿Cómo trata el scorer los empates y qué alternativa sería razonable?	6
8. ¿Qué afirmación causal no puede sostener sin comparar con un cross-encoder?	7
9. ¿Cómo seleccioné los checkpoints y qué variable permanece confundida?	7
10. ¿Qué limitaciones tiene Winoground como benchmark?	8
11. ¿Por qué un BLEU bajo en captions no implica que la descripción visual sea inútil?	8
12. ¿Qué experimento cerraría mejor la tesis del proyecto?	8
Sec. 11.5 — Preguntas y tareas de código	9
1. Dada una matriz 2x2, calcule manualmente text, image y group	9
2. Modifique el scorer para registrar empates y devolver tres estados	9
3. Diseñe un bootstrap estratificado por tag para el group score	10
4. Escriba un test parametrizado: text correcto solo, image correcto solo, group correcto, shape inválido	10
5. Complejidad temporal y espacial de evaluar N ejemplos con los embeddings ya calculados	11
Sec. 11.6 — Verificación del repositorio	11
1. Ejecute o explique make test y make validate, y qué evidencia producen	11
2. Ubique el scorer oficial, la prueba de ceguera y el análisis por tags	11
3. Muestre el commit o tag final y explique cómo Docker reproduce los resultados	11

Sec. 6 — Banco común (también puede caer)	12
Errores a evitar	12
Checklist de la mañana del examen	13

Cómo usar esta guía

No es un guion para memorizar. Cada pregunta trae cuatro cosas:

- **Qué evalúan en realidad** — casi ninguna pregunta busca la definición; buscan si entiendes por qué las cosas son así.
- **Respuesta** — lo que hay que decir, con los números exactos.
- **Evidencia** — el archivo y la línea que hay que abrir si te la piden. El examen permite consultar el repositorio.
- **La trampa** — la pregunta de seguimiento que viene después, y cómo no caer.

Dos reglas de la rúbrica que conviene tener presentes (Sec. 14.1 del documento del examen):

“Mencionar conceptos sin relacionarlos con decisiones concretas del proyecto recibe puntaje parcial.”

“Reconocer con precisión una limitación es preferible a improvisar una respuesta sin evidencia.”

Traducción práctica: **nunca respondas solo con teoría**. Toda respuesta debe aterrizar en un número, un archivo o una decisión concreta. Y si no sabes algo, dilo y explica cómo lo averiguarías — eso puntúa más que inventar.

Los números que hay que saber de memoria

Todo esto sale de `outputs/metrics/audit_modelos.csv` y `results/metricas.csv`.

Tabla principal — CLIP ViT-B/32 (laion2b), 400 pares oficiales

Métrica	Resultado	IC 95 %	Azar	Humano
text score	0.3475	[0.300, 0.398]	0.25	0.895
image score	0.1100	[0.080, 0.142]	0.25	0.885
group score	0.0750	[0.050, 0.102]	0.1667	0.855
GroupMatch	0.6750	[0.630, 0.720]	0.50	—

La frase de una línea: el único score cuyo intervalo supera el azar es text. image y group están **por debajo** del azar, y el humano está a 0.78 de distancia en group.

Los otros dos checkpoints

Modelo	text	image	group	GroupMatch
ViT-B/32 (laion2b)	0.3475	0.1100	0.0750	0.6750
ViT-B/16 (datacomp_xl)	0.2975	0.0875	0.0725	0.6425
ViT-L/14 (openai)	0.2875	0.1100	0.0850	0.6650

Ninguna diferencia entre ellos es significativa. McNemar da $p = 1.00$, 0.61 y 0.52 en group. Ver `outputs/metrics/audit_pareado.csv`.

Desglose por tag (ViT-B/32)

Tag	n	text	image	group
Relation	233	0.305	0.090	0.047
Object	141	0.390	0.106	0.085
Both	26	0.500	0.308	0.269

Las relaciones son lo más difícil, y son la mayoría del benchmark.

Prueba de ceguera

Condición	text	image	group	GroupMatch
Real	0.3475	0.1100	0.0750	0.6750
Imágenes permutadas	0.1350	0.0350	0.0150	0.4575

Retrieval vs composición (galería completa de 800)

Dirección	R@1	R@5	R@10	MRR
imagen -> texto	0.340	0.701	0.795	0.504
texto -> imagen	0.304	0.668	0.774	0.474

Contra group = 0.075. **Esa es la tesis del proyecto en dos números.**

Potencia estadística

Métrica	Diferencia mínima detectable (n=400, $\alpha=0.05$, potencia=0.80)
text	9.2 puntos
image	6.0 puntos
group	5.3 puntos
GroupMatch	9.4 puntos

Empates y márgenes (ViT-B/32)

- Empates exactos con $\text{atol}=0$: **cero** en las cuatro métricas.
- Mediana del margen de decisión: **0.0154**. Percentil 5: 0.00099.
- Rango de las similitudes: [0.0268, 0.4296], desviación 0.0583.
- Con $\text{atol}=1e-3$ aparecen 43 empates en text y el score baja de 0.3475 a 0.2925.

Síntesis inicial — 2 minutos (Sec. 2.1, vale 1 punto)

Léela en voz alta cronometrada hasta que salga sola. Debe cubrir cinco cosas: problema, pregunta, método, resultado principal y limitación.

El problema es que los modelos visión-lenguaje contrastivos se evalúan casi siempre con métricas de recuperación, y esas métricas pueden estar altas sin que el modelo entienda la estructura de la oración. Winoground aísla exactamente eso: cada ejemplo tiene dos imágenes y dos captions que comparten **el mismo conjunto de palabras en distinto orden**, así que el vocabulario es inútil por construcción y solo sirve la composición.

Mi pregunta experimental es: **¿el buen desempeño de CLIP en recuperación implica razonamiento composicional?** Es falsable — si lo implicara, un modelo con Recall@5 de 0.70 debería resolver los pares mínimos muy por encima del azar.

El método es evaluar OpenCLIP sobre los 400 pares oficiales con el scorer de Thrush et al., validado numéricamente contra los scores publicados, con intervalos bootstrap de 2000 rondas, desglose por tag y una prueba de ceguera que permuta las imágenes.

El resultado principal: con la galería completa CLIP recupera bien — Recall@5 de 0.70 imagen a texto — pero el group score es **0.075, por debajo del azar de 0.167**, contra 0.855 en humanos. La respuesta a mi pregunta es no.

La limitación más importante es que ese número depende de la métrica. Con GroupMatch, que pregunta si el emparejamiento correcto maximiza la similitud total en vez de exigir cuatro comparaciones simultáneas, el mismo modelo con las mismas similitudes obtiene **0.675 sobre un azar de 0.5**. Así que la afirmación honesta no es “CLIP no compone”, sino “CLIP falla el criterio estricto de Winoground, y una parte de esa caída es la exigencia de la métrica, no solo el modelo”.

Si te sobra tiempo, añade: “y con $n=400$ diferencias menores a 5.3 puntos en group no son detectables, así que comparar checkpoints por décimas no tiene sentido”.

Sec. 11.4 — Preguntas conceptuales, metodológicas y críticas

1. Defina text score, image score y group score sin leer el código

Qué evalúan: si entiendes que los nombres se refieren a **lo que se elige**, no a lo que se fija. Es un error clásico.

Respuesta. Cada ejemplo da una matriz 2x2 donde $\text{sim}[c][i]$ es la similitud entre el caption c y la imagen i .

- **text score:** se **fija la imagen** y el modelo debe elegir el **caption** correcto — en las dos direcciones. Vale 1 si $s(c0, i0) > s(c1, i0)$ y $s(c1, i1) > s(c0, i1)$.
- **image score:** se **fija el caption** y el modelo debe elegir la **imagen** correcta. Vale 1 si $s(c0, i0) > s(c0, i1)$ y $s(c1, i1) > s(c1, i0)$.
- **group score:** vale 1 solo si ambos valen 1. Las cuatro comparaciones a la vez.

En términos de la matriz: text compara **dentro de cada columna**, image **dentro de cada fila**, y group exige que cada elemento de la diagonal sea el máximo de su fila y de su columna.

Evidencia: `src/winoground_eval.py:1–35` (docstring) y las funciones en las líneas 195-215.

La trampa: “¿No suena al revés?” Sí, y hay que decirlo: la redacción verbal de algunos papers de seguimiento suena invertida. La referencia canónica es el código oficial, y lo validé numéricamente: mi scorer reproduce `text=0.3075, image=0.1050, group=0.0800` del `clip.jsonl` que el propio dataset publica.

2. ¿Por qué el azar del group score es 1/6?

Qué evalúan: si sabes que text e image **no son independientes**. La respuesta ingenua ($1/4 \times 1/4 = 1/16$) es la que quieren descartar.

Respuesta. No es 1/16 porque las dos métricas se calculan sobre **las mismas cuatro similitudes**, así que están correlacionadas.

La derivación limpia: bajo azar, las cuatro similitudes son cuatro valores distintos en orden aleatorio, y hay $4! = 24$ ordenaciones equiprobables. $\text{group} = 1$ exige que **ambas diagonales** superen a **ambas antidiagonales**. De las 24 ordenaciones, eso ocurre cuando las dos diagonales ocupan los dos primeros puestos: 2 formas de ordenar las diagonales entre sí $\times$ 2 formas de ordenar las antidiagonales entre sí = **4 ordenaciones favorables**.

$$P(\text{group} = 1) = \frac{4}{24} = \frac{1}{6} \approx 0.1667$$

Evidencia: `src/winoground_eval.py:26-33`, y el test `test_chance_levels_in_dict` en `tests/test_winoground_eval`

La trampa: “Entonces, ¿cuál es el azar de `GroupMatch`?” $\rightarrow 1/2$. `GroupMatch` solo pregunta cuál de las $2! = 2$ asignaciones posibles tiene mayor similitud total. Por eso **los dos números no son comparables entre sí** y siempre los reporto con su azar al lado.

3. ¿Cómo puede coexistir Recall@5 alto con group score bajo?

Qué evalúan: el corazón del proyecto.

Respuesta. Porque **son tareas de dificultad estructuralmente distinta**, y el atajo que resuelve una está prohibido en la otra.

En Recall@5 el modelo busca el caption correcto entre 800 candidatos que hablan de temas completamente distintos: perros, esquiadores, tazas, semáforos. Basta con acertar el **tema** — reconocer vocabulario y objetos salientes. Por eso obtengo $R@5 = 0.70$.

En Winoground los dos candidatos tienen **exactamente el mismo conjunto de palabras**. “an old person kisses a young person” y “a young person kisses an old person” tienen idéntica bolsa de palabras. El atajo léxico no solo es insuficiente: es **matemáticamente inútil**, porque un modelo que ignore el orden asigna necesariamente la misma puntuación a ambos. Lo único que distingue es el orden, es decir, la composición.

Dicho de otro modo: $R@K$ mide **discriminación entre temas**; el group score mide **discriminación dentro de un tema**. Un modelo puede ser excelente en lo primero y estar en el azar en lo segundo, y eso es exactamente lo que mido: 0.70 contra 0.075.

Evidencia: `outputs/metrics/recall_vs_group.json`, que guarda ambas cosas en el mismo archivo justamente para esta comparación.

La trampa: “¿No será que la galería de 800 es demasiado fácil?” Es una crítica válida y la reconozco: $R@K$ depende del tamaño y la diversidad de la galería, así que su valor absoluto no es interpretable por sí solo. Por eso el argumento **no** es “0.70 es alto”, sino que **las dos cifras se miden sobre las mismas 800 imágenes y los mismos 800 captions**, y aun así divergen en un factor de casi diez.

4. ¿Qué mide la prueba de ceguera y qué no mide?

Qué evalúan: si distingues un control negativo de una demostración positiva. Es la pregunta más fácil de sobrevender.

Respuesta. Lo que sí mide: que el modelo usa el contenido de la imagen. Al permutar las imágenes entre ejemplos, el group score cae de 0.075 a **0.015** y el text score de 0.3475 a 0.135. Si el modelo estuviera resolviendo la tarea con pistas no visuales — sesgos del lenguaje, longitud del caption, artefactos del scorer — permutar no cambiaría nada. Es un control que descarta una explicación alternativa concreta.

Lo que no mide: absolutamente nada sobre composición. Que el modelo *use* la imagen no dice *cómo* la usa. Un modelo que solo reconoce “hay una persona y una taza” también se derrumba al permutar. La prueba descarta el artefacto, no demuestra comprensión.

Evidencia: `src/blindness_probe.py`, resultados en `outputs/metrics/blindness.json`.

La trampa: “¿Por qué 0.015 y no 0?” Porque la permutación es aleatoria y algunos pares quedan casualmente compatibles — dos escenas de exterior con personas, por ejemplo. Además mi permutación es una *aproximación* a un desarreglo: el bucle de corrección en `src/blindness_probe.py:29–32` puede dejar algún punto fijo. Está documentado en el docstring.

5. ¿Por qué la permutación de imágenes no demuestra por sí sola razonamiento composicional?

Es la segunda mitad de la 4. **Respuesta corta:** porque es un control negativo, no un experimento positivo. Establece que el canal visual está vivo; el razonamiento composicional exigiría mostrar que el modelo distingue **dos escenas con los mismos objetos y distinta estructura relacional**, que es justo lo que el group score de 0.075 dice que no hace.

Analogía útil si te la piden: comprobar que a un paciente le funciona el oído no demuestra que entienda el idioma.

6. ¿Qué tipos de errores dominan y por qué son difíciles para un dual encoder?

Respuesta. Dominan los de **relación**. Por tag:

Tag	n	group
Relation	233	0.047
Object	141	0.085
Both	26	0.269

Las relaciones son casi la mitad del benchmark y tienen el peor score — la mitad que los ejemplos de objeto.

Por qué le cuesta a un dual encoder. CLIP produce **un solo vector global** por imagen y **un solo vector global** por texto, y los compara por coseno. La imagen nunca ve el texto. En esa arquitectura, “el perro persigue al gato” y “el gato persigue al perro” activan el mismo conjunto de conceptos; para separarlos habría que codificar el *rol* de cada entidad en la relación, y un vector promediado sobre toda la escena no tiene dónde guardar esa asignación de roles sin que se mezcle con el resto.

El entrenamiento tampoco lo exige: el objetivo contrastivo con negativos aleatorios de otro tema se resuelve perfectamente reconociendo el tema. **Nunca hay presión para aprender el orden**, porque ningún negativo del lote lo requiere.

Evidencia: `outputs/metrics/audit_por_tag.csv`, `src/error_analysis.py`.

La trampa: “¿Y el tag ‘Both’ no contradice tu tesis, con 0.269?” Buena observación, y la respuesta honesta es: **n = 26**. Con esa muestra el intervalo es enorme y no soporta ninguna conclusión. Lo reporto por completitud, no como evidencia.

7. ¿Cómo trata el scorer los empates y qué alternativa sería razonable?

Qué evalúan: Sec. 11.2.2. Aquí puedes lucirte porque el trabajo está hecho.

Respuesta en tres partes.

Cómo los trata el scorer oficial: con `>` estricto. Un empate exacto devuelve `False` y **se contabiliza en silencio como fallo**. Eso colapsa dos estados que son epistémicamente distintos: “el modelo prefiere la respuesta equivocada” y “el modelo no expresa ninguna preferencia”.

Qué hice: extendí el scorer a **tres estados** — correcto, incorrecto, empate — con una tolerancia `atol` configurable. La regla de combinación es que **un fallo estricto domina; en su ausencia, un empate deja el caso abierto**. Un empate nunca rescata a un fallo. Y añadí tres políticas para convertir el empate en número (`fail`, `half`, `pass`), que acotan el score verdadero por debajo y por arriba sin tener que elegir una convención.

Qué encontré — y esto es lo importante: **cero empates exactos** en los tres checkpoints. Así que la convención $>$ es empíricamente inocua. **Pero** eso no significa que las decisiones sean holgadas: la mediana del margen que decide cada comparación es **0.0154**, sobre similitudes que van de 0.027 a 0.430 con desviación 0.058. El 5 % de las decisiones se resuelve por menos de 0.001. Con una tolerancia de 0.001 aparecen 43 empates en text y el score cae de 0.3475 a 0.2925.

La conclusión honesta: no hay empates, pero el score es **frágil** ante cualquier cambio de preprocesado que mueva las similitudes en la tercera decimal.

La alternativa razonable: **GroupMatch** (Zhu et al., ICLR 2026). En vez de exigir cuatro comparaciones simultáneas, pregunta si el emparejamiento correcto maximiza la similitud total: $s_{00} + s_{11} > s_{01} + s_{10}$. Azar 1/2. El mismo modelo pasa de **0.075 a 0.675**.

Evidencia: src/winoground_eval.py (constantes CORRECTO/INCORRECTO/EMPATE), docs/adr/0003-empates-y-metrica-alternativa.md, outputs/metrics/audit_empates.csv, outputs/metrics/audit_margenes y 26 tests en tests/test_ties_and_groupmatch.py.

La trampa: “Si GroupMatch da 0.675, ¿entonces CLIP sí compone?” No, y hay que ser preciso. Son preguntas distintas: el group score mide si el modelo ordena bien **cada comparación por separado**; GroupMatch, si acierta la **asignación global**. Que acierte la asignación global el 67 % del tiempo con un azar del 50 % es mejor que el azar, pero sigue muy lejos del 85.5 % humano bajo el criterio estricto. Lo que la brecha entre ambas métricas demuestra es que **parte de lo que se reporta como “fallo composicional” es exigencia de la métrica**, no que el fallo no exista.

8. ¿Qué afirmación causal no puede sostener sin comparar con un cross-encoder?

Qué evalúan: Sec. 11.2.1. Honestidad metodológica. **Esta es la pregunta donde más se gana o se pierde.**

Respuesta. No puedo sostener que CLIP falla **porque** carece de atención cruzada.

Mi evidencia es correlacional: observo que un modelo sin atención cruzada falla. Pero CLIP se distingue de un cross-encoder en muchas cosas a la vez — arquitectura, objetivo de entrenamiento, datos, escala. Cualquiera de ellas podría ser la causa. Afirmar el mecanismo exigiría **variar solo la atención cruzada y dejar todo lo demás fijo**.

Eso es exactamente lo que estoy montando con BlipForImageTextRetrieval, que expone **dos cabezas sobre el mismo cuerpo**: use_itm_head=False da una similitud coseno entre proyecciones separadas (dual-encoder, la imagen nunca ve el texto), y use_itm_head=True pasa por atención cruzada real entre los tokens de texto y los parches de imagen. Mismos pesos, mismo preprocesado, mismos datos, mismo hardware. **La única variable que cambia es la presencia de atención cruzada.**

Evidencia: src/blip_utils.py y scripts/12_run_blip.py.

Si te preguntan por los resultados y aún no están: dilo directamente — “el experimento está implementado y corriendo; el entorno tiene un problema de I/O que hace que cada arranque con torch cueste varios minutos, y lo documenté en results/configuracion_experimental.json. Sin ese resultado, mi afirmación se queda en correlacional y así lo digo en el informe.” Eso puntúa; inventar un número, no.

La trampa: “¿Y si el cross-encoder también falla?” Sería un resultado **más** interesante, no un fracaso: significaría que el problema no es la arquitectura sino el paradigma de entrenamiento, y apuntaría hacia el objetivo contrastivo con negativos fáciles. Mi hipótesis es falsable en las dos direcciones, y eso es lo que la hace una hipótesis.

9. ¿Cómo seleccioné los checkpoints y qué variable permanece confundida?

Respuesta. Los tres checkpoints son ViT-B-32/laion2b, ViT-B-16/datacomp_xl y ViT-L-14/openai, elegidos por ser los del Cuaderno 10 y por caber en memoria en MPS.

La variable confundida es grave y hay que declararla: cambian **a la vez el tamaño del modelo y los datos de preentrenamiento**. B/32 se entrenó con LAION-2B, B/16 con DataComp-XL y L/14 con el conjunto propietario

de OpenAI. Así que **no puedo atribuir ninguna diferencia al tamaño ni a los datos por separado**. Para aislar el tamaño necesitaría la misma familia de datos en tres escalas.

Pero hay algo más importante: la pregunta es discutible porque **ninguna diferencia entre los tres es significativa**. McNemar sobre group da $p = 1.00, 0.61$ y 0.52 . Y la diferencia mínima detectable con $n=400$ es de **5.3 puntos**, mientras que la mayor diferencia observada es de 1.25 puntos. Es decir: aunque hubiera desconfundido las variables, **el benchmark no tiene potencia para resolverlas**.

Evidencia: configs/checkpoints.yaml, outputs/metrics/audit_pareado.csv, outputs/metrics/audit_potencia

10. ¿Qué limitaciones tiene Winoground como benchmark?

Respuesta, en orden de importancia:

1. **Tamaño.** $n = 400$. La diferencia mínima detectable en group es de 5.3 puntos, así que casi toda la literatura que compara modelos por uno o dos puntos está reportando ruido.
2. **No mide solo composición.** Diwan et al. (EMNLP 2022) reanotaron los 400 ítems y encontraron que 38 exigen detectar rasgos visuales sutiles, 56 tienen imágenes inusuales, 50 captions difíciles de parsear y 46 son **ambiguos**. Una parte del fallo es ruido del dataset, no incapacidad del modelo.
3. **La métrica es una elección, no un hecho.** Ya lo demostré: 0.075 contra 0.675 según el criterio.
4. **Solo inglés.** No dice nada sobre lenguas con orden más flexible o marcado morfológico de roles.
5. **Techo humano medido bajo el criterio estricto.** El 0.855 humano no es recomputable bajo GroupMatch, porque el paper publicó acuerdo humano y no las cuatro similitudes por ejemplo. Por eso mi tabla deja esa celda vacía en vez de rellenarla.

Lo que sí conserva su valor: por construcción es inmune al ataque que hundió a otros benchmarks composicionales. SugarCrepe (NeurIPS 2023) mostró que en ARO, CREPE y VL-CheckList un modelo **ciego**, sin acceso a la imagen, supera al estado del arte, porque los negativos generados por plantilla eran distinguibles por fluidez del texto. En Winoground eso es imposible: los dos captions tienen el mismo conjunto de palabras.

11. ¿Por qué un BLEU bajo en captions no implica que la descripción visual sea inútil?

Respuesta. Porque BLEU mide **solapamiento léxico n-grama con una referencia**, no si la descripción es correcta. “un perro corre por la playa” y “un can se desplaza sobre la arena” describen la misma escena con BLEU cercano a cero.

En mi caso BLIP obtuvo BLEU 0.073 y ROUGE-L 0.191. Lo que eso mide es que BLIP usa un vocabulario distinto al de las referencias de Winoground, que además están escritas para ser pares mínimos y no descripciones naturales. Un caption puede ser **visualmente correcto y puntuar bajo**.

La lección general, que es la misma que la del proyecto entero: **una métrica que se interpreta sola engaña**. BLEU necesita acompañarse de evaluación humana o de una métrica basada en semántica; igual que R@K necesita acompañarse del group score.

12. ¿Qué experimento cerraría mejor la tesis del proyecto?

Respuesta. El de la pregunta 8: **BLIP ITC contra BLIP ITM sobre los mismos 400 pares**. Es el único que convierte la afirmación mecanística en un experimento controlado, porque varía la atención cruzada dejando todo lo demás constante.

Predicción falsable, para que quede claro que no es una expectativa vaga: si la atención cruzada es la causa, ITM debería subir el group score **por encima del azar de 0.167**, mientras ITC se queda al nivel de CLIP. Si ambos se quedan abajo, la causa no es la arquitectura sino el objetivo contrastivo, y la tesis del proyecto habría que reformularla.

El segundo mejor, si hubiera más tiempo: filtrar los 46 ítems que Diwan et al. marcaron como ambiguos y re-medir. Separaría el fallo del modelo del ruido del benchmark.

Sec. 11.5 — Preguntas y tareas de código

1. Dada una matriz 2x2, calcule manualmente text, image y group

Practica con esta, que es el ejemplo 0 real del benchmark (an old person kisses a young person):

$$\text{sim} = \begin{pmatrix} 0.343 & 0.329 \\ 0.325 & 0.320 \end{pmatrix}$$

donde $\text{sim}[c][i]$, es decir $\text{sim}[0][0] = s(c0, i0) = 0.343$.

text score — fija la imagen, compara captions (columnas):

- Imagen 0: ¿ $s(c0, i0) > s(c1, i0)$? -> $0.343 > 0.325$ -> **sí**
- Imagen 1: ¿ $s(c1, i1) > s(c0, i1)$? -> $0.320 > 0.329$ -> **no**
- -> **text = 0**

image score — fija el caption, compara imágenes (filas):

- Caption 0: ¿ $s(c0, i0) > s(c0, i1)$? -> $0.343 > 0.329$ -> **sí**
- Caption 1: ¿ $s(c1, i1) > s(c1, i0)$? -> $0.320 > 0.325$ -> **no**
- -> **image = 0**

group = 0 AND 0 = 0.

GroupMatch: $0.343 + 0.320 = 0.663$ contra $0.329 + 0.325 = 0.654$. Margen +0.009 -> **acierta**. Este ejemplo ilustra perfectamente la brecha entre métricas.

2. Modifique el scorer para registrar empates y devolver tres estados

Ya está hecho. Muéstralo en `src/winoground_eval.py`:

CORRECTO, INCORRECTO, EMPATE = "correcto", "incorrecto", "empate"

```
def _compare(a: float, b: float, atol: float) -> int:
    """+1 si a supera a b, -1 si b supera a a, 0 si empatan dentro de `atol`."""
    if a - b > atol:
        return 1
    if b - a > atol:
        return -1
    return 0

def _combine(*comparisons: int) -> str:
    """Un fallo estricto domina; en su ausencia, un empate deja el caso abierto."""
    if any(c < 0 for c in comparisons):
        return INCORRECTO
    if any(c == 0 for c in comparisons):
        return EMPATE
    return CORRECTO

def text_status(sim, atol: float = 0.0) -> str:
    sim = _as_matrix(sim)
    return _combine(
        _compare(sim[0, 0], sim[1, 0], atol),
        _compare(sim[1, 1], sim[0, 1], atol),
    )
```

Lo que hay que explicar al mostrarlo:

- Por qué la regla es “el fallo domina”: un empate no puede rescatar a un fallo, porque el fallo es evidencia de preferencia equivocada y el empate es ausencia de evidencia.
- Por qué `atol` vale 0.0 por defecto: para que el comportamiento coincida exactamente con el scorer oficial y los números sigan siendo comparables con la literatura.
- Cómo demostré que no rompí nada: `test_estados_coinciden_con_el_scorer_oficial_sin_tolerancia` compara ambos caminos sobre 200 matrices aleatorias y exige `(status == CORRECT0) is scorer_oficial(sim)`.

3. Diseñe un bootstrap estratificado por tag para el group score

También está hecho, en `src/metrics.py::stratified_bootstrap_ci`:

```
grupos = [np.flatnonzero(strata_arr == s) for s in np.unique(strata_arr)]
for r in range(rounds):
    muestra = np.concatenate([g[rng.integers(0, len(g), size=len(g))] for g in grupos])
    means[r] = vals[muestra].mean()
```

La justificación, que es lo que evalúan: el bootstrap simple trata los 400 ejemplos como intercambiables y remuestra libremente. Pero los tags están desbalanceados (Relation 233, Object 141, Both 26) y el group score difiere mucho entre ellos (0.047 contra 0.269). Un remuestreo libre hace fluctuar **la composición** de la muestra además de su contenido, e infla el intervalo con una fuente de variación que no nos interesa: no preguntamos qué pasaría si el benchmark tuviera otra mezcla de tags, sino qué pasaría con otros ejemplos de la misma mezcla.

Fijando el tamaño de cada estrato, el intervalo refleja solo la incertidumbre dentro de cada tipo de ejemplo.

4. Escriba un test parametrizado: text correcto solo, image correcto solo, group correcto, shape inválido

```
import numpy as np
import pytest
from src.winoground_eval import text_correct, image_correct, group_correct, per_example_scores

@pytest.mark.parametrize("sim, esperado_text, esperado_image, esperado_group", [
    # Diagonal dominante: los cuatro contrastes salen bien.
    (np.array([[0.9, 0.1], [0.1, 0.9]]), True, True, True),
    # text sí, image no: la fila 0 pierde contra la columna.
    (np.array([[0.9, 0.95], [0.1, 0.97]]), True, False, False),
    # image sí, text no.
    (np.array([[0.9, 0.1], [0.95, 0.97]]), False, True, False),
    # Antidiagonal dominante: todo falla.
    (np.array([[0.1, 0.9], [0.9, 0.1]]), False, False, False),
])
def test_scorer_parametrizado(sim, esperado_text, esperado_image, esperado_group):
    assert text_correct(sim) is esperado_text
    assert image_correct(sim) is esperado_image
    assert group_correct(sim) is esperado_group

@pytest.mark.parametrize("forma", [(2, 3), (3, 2), (2,), (2, 2, 2)])
def test_shape_invalido(forma):
    with pytest.raises(ValueError):
        per_example_scores([np.zeros(forma)])
```

El detalle que suma: `is True` en vez de `== True` obliga a que el scorer devuelva un `bool` de Python y no un `np.bool_`. Por eso las funciones envuelven el resultado en `bool(...)`.

5. Complejidad temporal y espacial de evaluar N ejemplos con los embeddings ya calculados

Respuesta. Con embeddings de dimensión d ya calculados:

- **Tiempo:** cada ejemplo requiere una matriz 2×2 , que es un producto $(2 \times d) \cdot (d \times 2) = O(d)$ por ejemplo, más $O(1)$ comparaciones. Total $O(N \cdot d)$. Con $N=400$ y $d=512$ son unos 400 000 productos: milisegundos.
- **Espacio:** $O(N \cdot d)$ para los embeddings — en mi caso 800×512 float32 ~ 1.6 MB — y $O(N)$ para los scores. Las matrices 2×2 son 400×4 float64 ~ 12 KB.

El contraste que hay que hacer, porque es la respuesta interesante: un **cross-encoder no factoriza**. No hay embeddings que cachear, porque la representación depende del par. Cada ejemplo exige **4 pasadas completas** por la red, y el benchmark 1600. Esa asimetría — $O(N+M)$ contra $O(N \cdot M)$ — es precisamente la razón por la que los sistemas de retrieval en producción usan dual-encoders para recuperar y reservan el cross-encoder para reordenar los primeros candidatos.

Lo aproveché en el diseño: `scripts/10_export_sims.py` vuelca las matrices 2×2 a disco, y `scripts/11_metric_audit.py` recalcula **todas** las métricas en segundos sin importar torch.

Sec. 11.6 — Verificación del repositorio

1. Ejecute o explique `make test` y `make validate`, y qué evidencia producen

- **`make test`** -> `pytest -q. 42 tests`: 16 originales del scorer y las métricas, más 26 nuevos de empates y GroupMatch. Producen la garantía de que el scorer sigue coincidiendo con el oficial.
- **`make validate`** -> `scripts/validate_against_official.py`, que compara mi scorer contra `statistics/model_scores/clip.jsonl`, el archivo de scores por ejemplo que publica el propio dataset. Es la validación externa: no me creo mi scorer porque pase mis tests, sino porque reproduce números publicados.

2. Ubique el scorer oficial, la prueba de ceguera y el análisis por tags

Qué	Dónde
Scorer oficial	<code>src/winoground_eval.py</code> — <code>text_correct</code> , <code>image_correct</code> , <code>group_correct</code>
Tres estados y GroupMatch	<code>src/winoground_eval.py</code> — <code>text_status</code> , <code>group_match</code> , <code>tie_report</code>
Prueba de ceguera	<code>src/blindness_probe.py</code> — <code>run_blindness_probe</code>
Análisis por tags	<code>src/error_analysis.py</code> — <code>scores_by_tag</code>
Estadística	<code>src/metrics.py</code> — <code>bootstrap_ci</code> , <code>stratified_bootstrap_ci</code> , <code>mcnemar_exact</code> , <code>minimum_resolvable_difference</code>
Cross-encoder	<code>src/blip_utils.py</code>

3. Muestre el commit o tag final y explique cómo Docker reproduce los resultados

Tag: `final-mcc225`. **Repositorio evaluado:** github.com/nielspac177/MCC225-Winoground.

Docker: el Dockerfile parte de `python:3.12-slim`, instala desde `pyproject.toml`, fija `HF_HOME` dentro del workspace y ejecuta la secuencia `01_prepare_data.py` -> `02_run_winoground.py` -> `03_make_figures.py`. `docker-compose.yml` monta el repo y pasa `HF_TOKEN`.

Y aquí hay que declarar una limitación conocida, sin esperar a que la encuentren. `02_run_winoground.py` se interbloquea en su bloque final, el de FAISS: carga el runtime OpenMP de FAISS y el de PyTorch en el mismo proceso, y en macOS eso produce un deadlock — el proceso queda a 0 % de CPU indefinidamente. Todas las métricas anteriores a ese bloque se escriben correctamente; lo que no termina es la demo de FAISS.

Cómo lo sorteé: separando el cómputo del análisis. `scripts/10_export_sims.py` vuelca las matrices 2x2 y `scripts/11_metric_audit.py` hace todo el análisis sin importar ni torch ni FAISS. Está documentado en `results/configuracion_experimental.json`, campo `limitaciones_conocidas`.

Decir esto **antes** de que lo pregunten demuestra que conoces tu propio pipeline. Ocultarlo y que lo descubran es el peor escenario posible.

Sec. 6 — Banco común (también puede caer)

Sec. 6.1.7 — ¿Qué limitación tiene un dual encoder para interacciones finas? No hay ningún punto del cómputo donde un token de texto pueda consultar una región concreta de la imagen. La imagen se comprime a un vector **antes** de ver el texto, así que cualquier información que no sobreviva a esa compresión se pierde irreversiblemente. Para “el perro a la izquierda del gato” haría falta ligar cada entidad a su posición, y eso exige interacción token-región. Mis 0.047 de group en ejemplos de relación son la medición de ese límite.

Sec. 6.1.5 — ¿Diferencia entre self-attention y cross-attention en su proyecto? En CLIP solo hay **self-attention**: los parches se atienden entre sí dentro del encoder visual, y los tokens entre sí dentro del textual. Las dos torres nunca se comunican; el único punto de contacto es el producto punto final entre dos vectores. En la cabeza ITM de BLIP hay **cross-attention**: las consultas vienen de los tokens de texto y las claves y valores de los parches de imagen, así que cada palabra puede mirar regiones concretas. Ese es el contraste que aísla mi ablación.

Sec. 6.2.2 — ¿Qué experimento rechazaría su hipótesis? Mi hipótesis es que el buen retrieval no implica composición. La rechazaría un modelo con R@5 alto que además obtuviera group score cerca del techo humano. También la debilitaría seriamente que, al filtrar los ítems ambiguos de Diwan et al., el group score subiera hasta acercarse al humano: significaría que el fallo era del benchmark y no del modelo.

Sec. 6.2.5 — ¿La mejora es estable, significativa y generalizable, o solo descriptiva? Aquí no reporto ninguna mejora, sino un límite. Y sobre su estabilidad: es **significativo** que group esté por debajo del azar (IC [0.050, 0.102] contra 0.1667), pero **no es significativa** ninguna diferencia entre checkpoints (McNemar $p \geq 0.52$). Sobre generalización, la limitación es que 400 ejemplos en inglés de un dominio fotográfico concreto no autorizan a extrapolar a otros idiomas ni dominios.

Sec. 6.2.8 — ¿Qué afirmación no puede sostener con sus resultados? Tres, en orden de tentación: (1) que CLIP “no entiende composición” — solo puedo decir que falla este criterio en este benchmark; (2) que la falta de atención cruzada es la **causa** — es correlacional hasta que cierre la ablación de BLIP; (3) que un checkpoint sea mejor que otro — no hay potencia estadística para eso.

Sec. 6.3.4 — ¿Qué ocurre si los embeddings no se normalizan antes del producto punto? El producto punto deja de ser el coseno y pasa a incluir las magnitudes. Como Winoground compara **la misma imagen contra dos captions**, una diferencia de norma entre los dos captions sesgaría sistemáticamente la comparación en favor del de mayor norma, sin relación con la semántica. En `src/openclip_utils.py` normalizo con L2 en `encode_images` y `encode_texts` justo por eso.

Errores a evitar

1. **No digas “CLIP no entiende composición”.** Di “CLIP falla el criterio estricto de Winoground; con Group-Match el mismo modelo obtiene 0.675, así que parte de la caída es la métrica”.
2. **No presentes la prueba de ceguera como evidencia de composición.** Es un control negativo.

3. **No compares los tres checkpoints como si uno fuera mejor.** Ninguna diferencia es significativa, y esa es la respuesta correcta.
 4. **No inventes el resultado de BLIP si aún no está.** Explica el diseño y di que está corriendo.
 5. **No mezcles group score con GroupMatch sin decir el azar de cada uno** (1/6 contra 1/2).
 6. **Declara el deadlock de FAISS antes de que lo pregunten.**
-

Checklist de la mañana del examen

- ☐ `git log -1` y `git tag` — saber de memoria el hash corto y que el tag es `final-mcc225`.
- ☐ Confirmar que `github.com/nielspace177/MCC225-Winoground` está actualizado.
- ☐ Tener abiertos: `src/winoground_eval.py`, `outputs/metrics/audit_modelos.csv`, `results/metrics.csv`.
- ☐ `.venv/bin/python -m pytest -q` — verde.
- ☐ Ensayar la síntesis de 2 minutos con cronómetro.
- ☐ Memorizar cuatro números: **0.3475 / 0.1100 / 0.0750** contra azar **0.1667**, y **GroupMatch 0.675**.